

Failure handling (three errors, three reactions) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Failure handling` section — the same file also holds the [Basic exercise](#), under a `# Basic` section. Run it with `cd practice && python custom_errors_practice.py`.

Builds on: the `# Basic` section of that same file — same habit, three error classes instead of one. Nothing to copy or import: scroll up in the file you already have.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need 3 different custom error types, one function that can raise any of the 3 depending on what happened, and a piece of code that catches each one separately — not all together in one block. Each error should get its own, different reaction.

Think of a real example: placing an order can fail because the item is out of stock, the payment was declined, or the address is wrong — three different problems that should be handled differently.

Here are the exact pieces you need:

- Make 3 classes, each one built from `Exception`: `OutOfStockError`, `PaymentDeclinedError`, `InvalidAddressError`.
- One function takes an input (like a word describing what went wrong) and raises the matching error.
- A `try` block can have several `except` parts, checked one after another.
- Each `except` part should actually do something different — not just print a different word.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You need 3 different custom error classes, one function that can raise any of the 3 depending on its input, and a caller with 3 *separate* `except` blocks — not one shared one. The whole point is that each error type gets its own, different reaction, not just its own printed message.

Think of a real example: a function processing an order might fail because the item is out of stock, the payment was declined, or the address is invalid — three genuinely different problems that a caller should handle differently.

- Define 3 classes, each inheriting from `Exception`: e.g. `OutOfStockError`, `PaymentDeclinedError`, `InvalidAddressError`.
- One function takes an input (like a status code, or a string describing what went wrong) and raises the matching error.
- A `try/except` block can have multiple `except` clauses, checked in order: `except OutOfStockError: ... except PaymentDeclinedError: ... except InvalidAddressError: ...`
- Each `except` block should do something *different* — not just print a different string, but actually react differently (like retrying, versus asking the user to fix something, versus just logging and stopping).

Difference between Basic and Intermediate: Basic gives you the ingredients in plain words.

Intermediate shows the exact syntax — 3 classes, 3 raises, 3 `except` blocks, and explains what each piece of syntax does — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make 3 error types: OutOfStockError, PaymentDeclinedError, InvalidAddressError
```

```
function process_order(problem_type):  
    if problem_type is "stock": raise OutOfStockError  
    if problem_type is "payment": raise PaymentDeclinedError  
    if problem_type is "address": raise InvalidAddressError
```

```
for each of the 3 problem types:  
    try process_order with that type  
    except OutOfStockError: say "try again later, tell the warehouse"  
    except PaymentDeclinedError: say "ask for a different payment method"  
    except InvalidAddressError: say "ask the customer to fix their address"
```

```
# practice/custom_errors_practice.py – Failure handling section  
class OutOfStockError(Exception):  
    pass  
  
class PaymentDeclinedError(Exception):  
    pass  
  
class InvalidAddressError(Exception):  
    pass  
  
def process_order(problem_type):  
    if problem_type == "stock":  
        raise OutOfStockError("item is out of stock")  
    # add the other two raises yourself
```

Try finishing the function and the 3 `except` parts yourself first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```

define OutOfStockError, PaymentDeclinedError, InvalidAddressError,
    each as a kind of Exception

function process_order(problem_type):
    if problem_type is "stock":
        raise OutOfStockError
    if problem_type is "payment":
        raise PaymentDeclinedError
    if problem_type is "address":
        raise InvalidAddressError

for each of the 3 problem types:
    try to process_order with that type
    except OutOfStockError: print "try again later, notify the warehouse"
    except PaymentDeclinedError:
        print "ask the customer for a different payment method"
    except InvalidAddressError: print "ask the customer to fix their address"

```

```

# practice/custom_errors_practice.py – Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    # add the other two raises yourself

```

Try finishing the function and the 3-branch `except` block yourself before checking the [Solution](#).

Difference between Basic and Intermediate: same underlying idea (3 error types, 3 different reactions) at 2 completeness levels, shown here both as pseudocode and as near-complete code — Basic proves the concept works, Intermediate adds the type contract real Python expects.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)